



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

The JAVA Sandbox, Virtual Machines

materials are based on security courses at other institutions

Separation Design Space



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **In what dimension should objects be separated?**
 - Physical, temporal, logical, cryptographic
- **To what degree should objects be kept separate?**
 - Complete isolation, ..., limited sharing, usage control, full sharing
- **What are the objects that will be kept apart?**
 - Parts of programs, programs, sets of programs, OS environments
- **What mechanisms will provide separation?**
 - Memory protection, sandboxes, virtual machines, ...

Separation Design Space



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **In what dimension should objects be separated?**
 - Physical, temporal, **logical**, cryptographic
- **To what degree should objects be kept separate?**
 - **Complete isolation, ..., limited sharing, usage control, full sharing**
- **What are the objects that will be kept apart?**
 - Parts of programs, **programs, sets of programs, OS environments**
- **What mechanisms will provide separation?**
 - Memory protection, **sandboxes, virtual machines, ...**

Buffer Overflow in C



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Why buffer overflow can happen?
 - Design flaw in C?
 - Design flaw in calling convention?
 - Design flaw in x86 architecture?

Discussion 5 mins (bonus question)

- Java provides better security.
 - Better isolation.

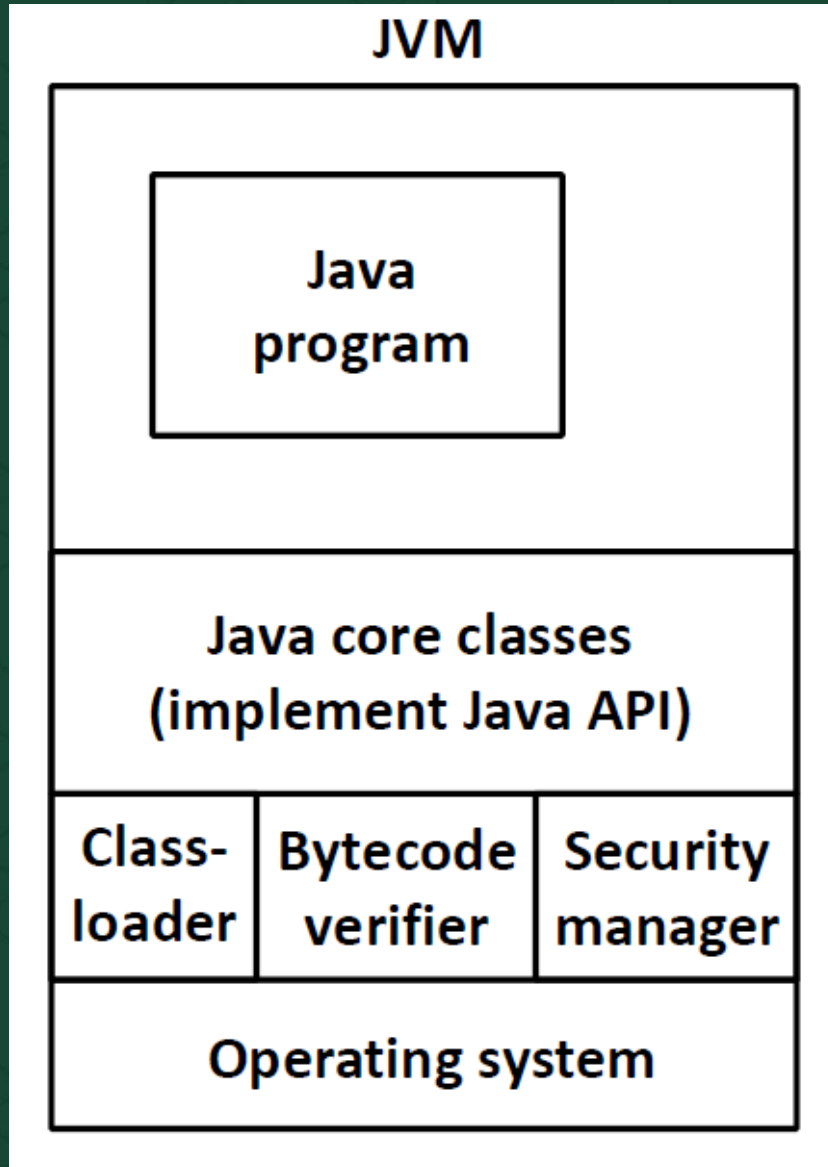


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Sandbox #1: The Java Virtual Machine (**JVM**)

- **Java core classes** are the fundamental **runtime libraries** that implement the Java API (e.g., java.lang, java.util, java.io, etc.).
- **JRE** (Java Runtime Environment) = **JVM** + **core classes**
- The **JDK** includes **much more** than just the core classes — it contains the **compiler** (javac), debugger, development tools, and the JRE **JDK** = **JRE** + **developer tools**

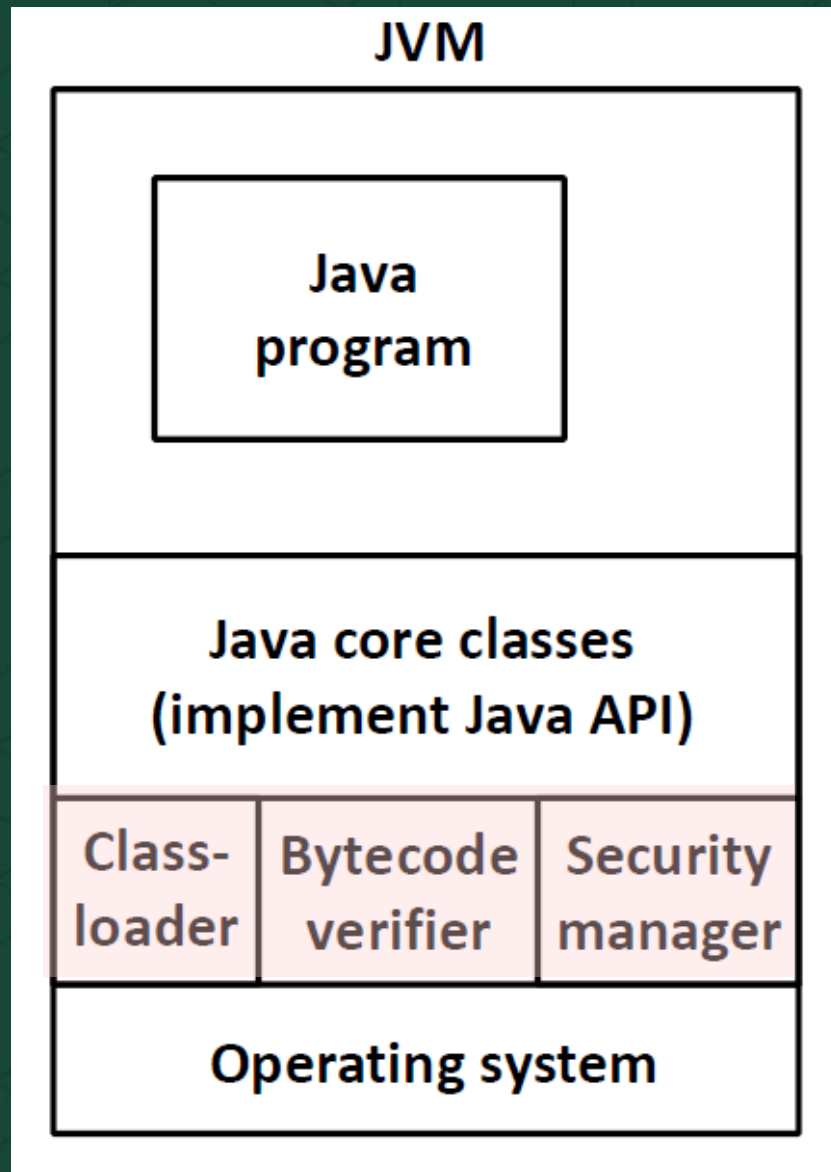
Sandbox #1: The Java Virtual Machine (JVM)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Java programs use the Java API
 - Can't run on or directly interact with the OS
- Java core classes interface between program and OS/hardware

Sandbox #1: The Java Virtual Machine (JVM)

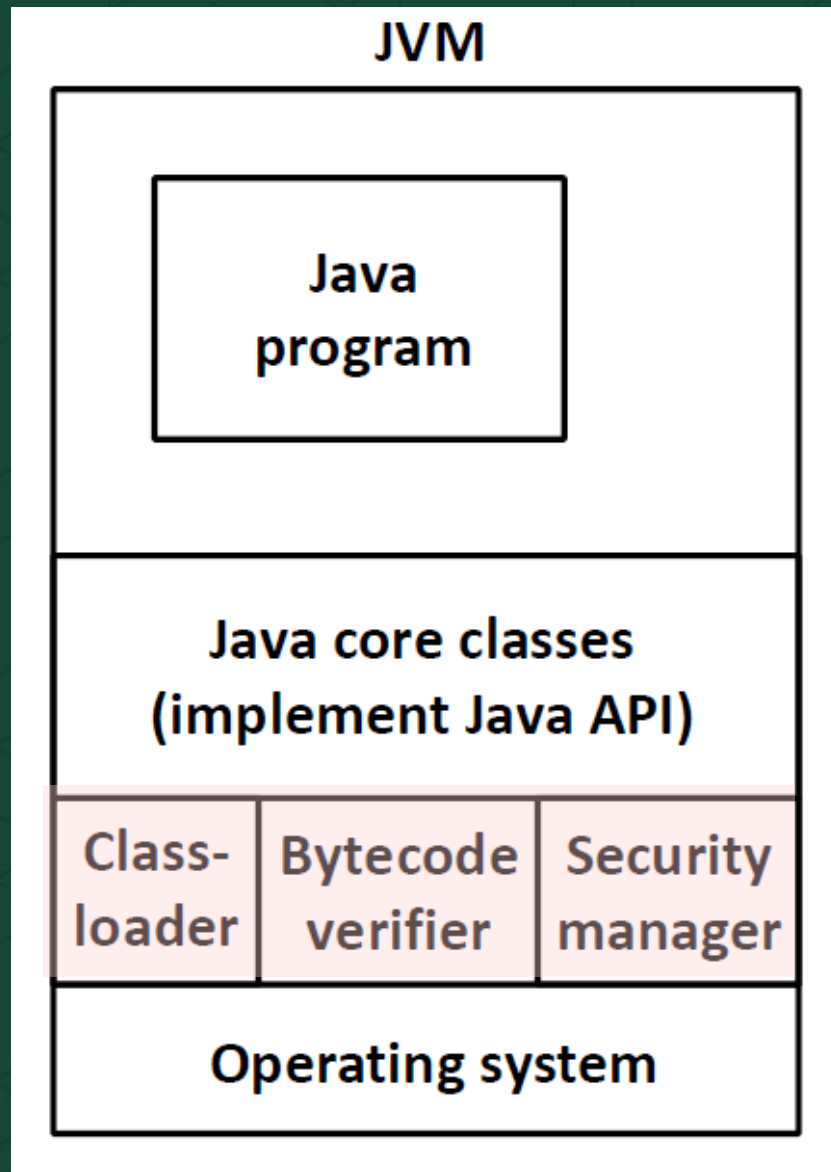


- (1) **Classloader** – loads classes into memory, ensures namespaces are separated;
- (2) **Bytecode Verifier** – checks code for type-safety, memory access, and illegal instructions before execution;
- (3) **Security Manager** – enforces runtime access control, preventing unauthorized file/network/system access.



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Sandbox #1: The Java Virtual Machine (JVM)



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Security manager mediates programs' attempts to reach outside the sandbox
- Classloader and bytecode verifier ensure that programs don't:
 - Corrupt each other
 - Corrupt the security manager
- Corrupt = interact with in ways not allowed by the API

Type Safety



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Java programs are composed of classes**
 - A *class* defines a collection of data fields and functions (methods) that operate on those fields
 - Instances of classes are *objects*
- **Type safety**
 - Only objects for which the program has a reference can be accessed
 - References cannot be forged
 - Memory cannot be accessed directly
 - A program can perform an operation on an object only if that operation is valid for that object
 - E.g., can't dereference an integer, can't read beyond end of array
- **Type safety is the cornerstone of Java security**

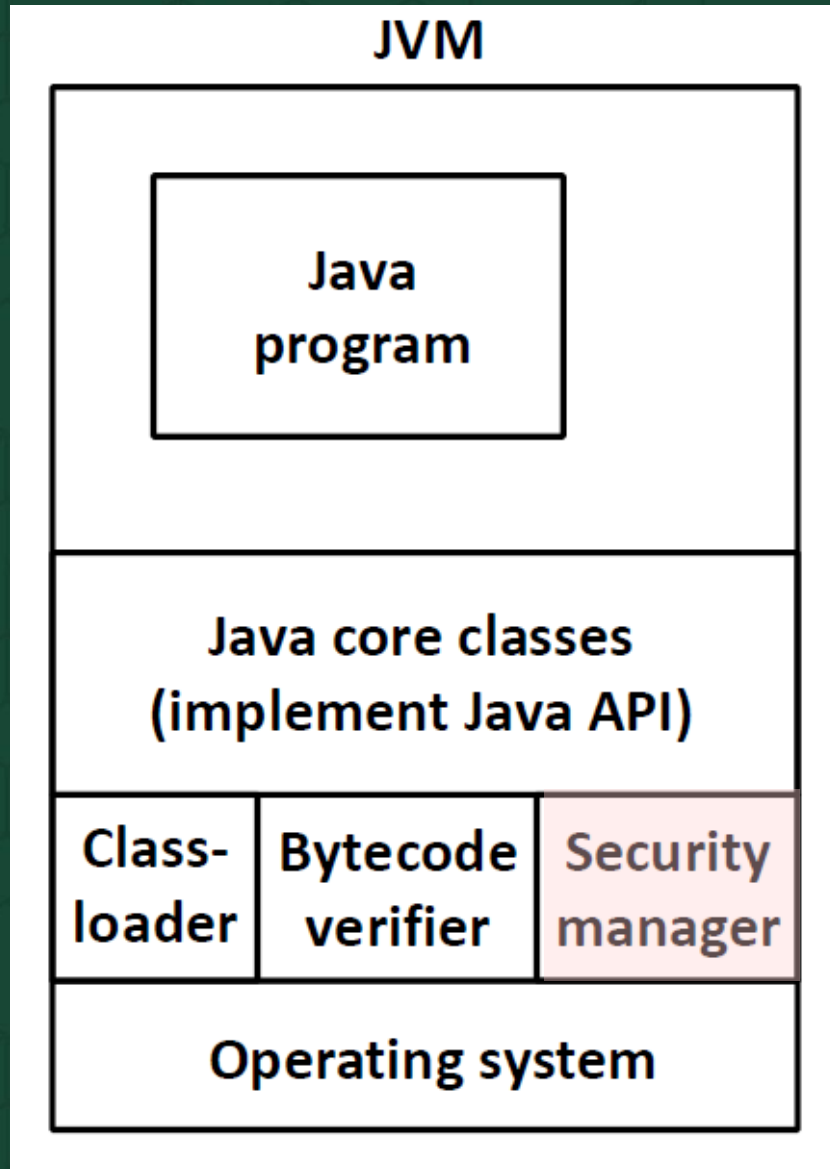
Is C type-safe?

```
void function() {  
    unsigned char a = 1;  
    unsigned char b = 2;  
    *(&b-1) = 10;  
    *(&b+1) = 11;  
    *(&b+2) = 12;  
    *(&b+3) = 13;  
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Security Manager



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Security manager essentially is a reference monitor**
 - Decides if a Java program's requested operation should be allowed
- **When a Java program makes a potentially dangerous request**
 - Java API code invokes the Security Manager
 - Security Manager throws a Security Exception if operation is denied
 - If operation is permitted, the Security Manager returns without exception
 - API code performs requested operation

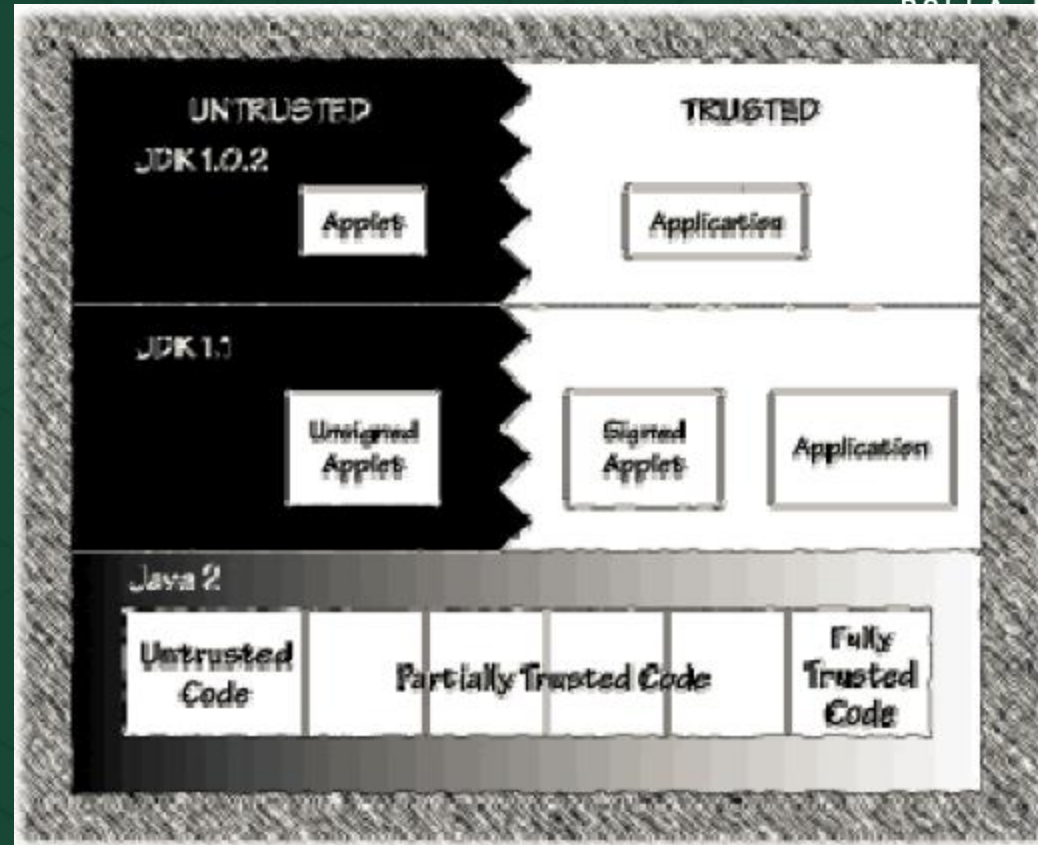
Security Manager Trust Model



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

ROLLA, MISSOURI

- Trust model has become increasingly refined
- **JDK 1.0.2**
 - Untrusted = any applet
 - Trusted = application
- **JDK 1.1**
 - Untrusted = unsigned applet
 - Trusted = signed applet or application
- **JDK 1.2 and beyond**
 - Many shades of gray



Security Manager Trust Model

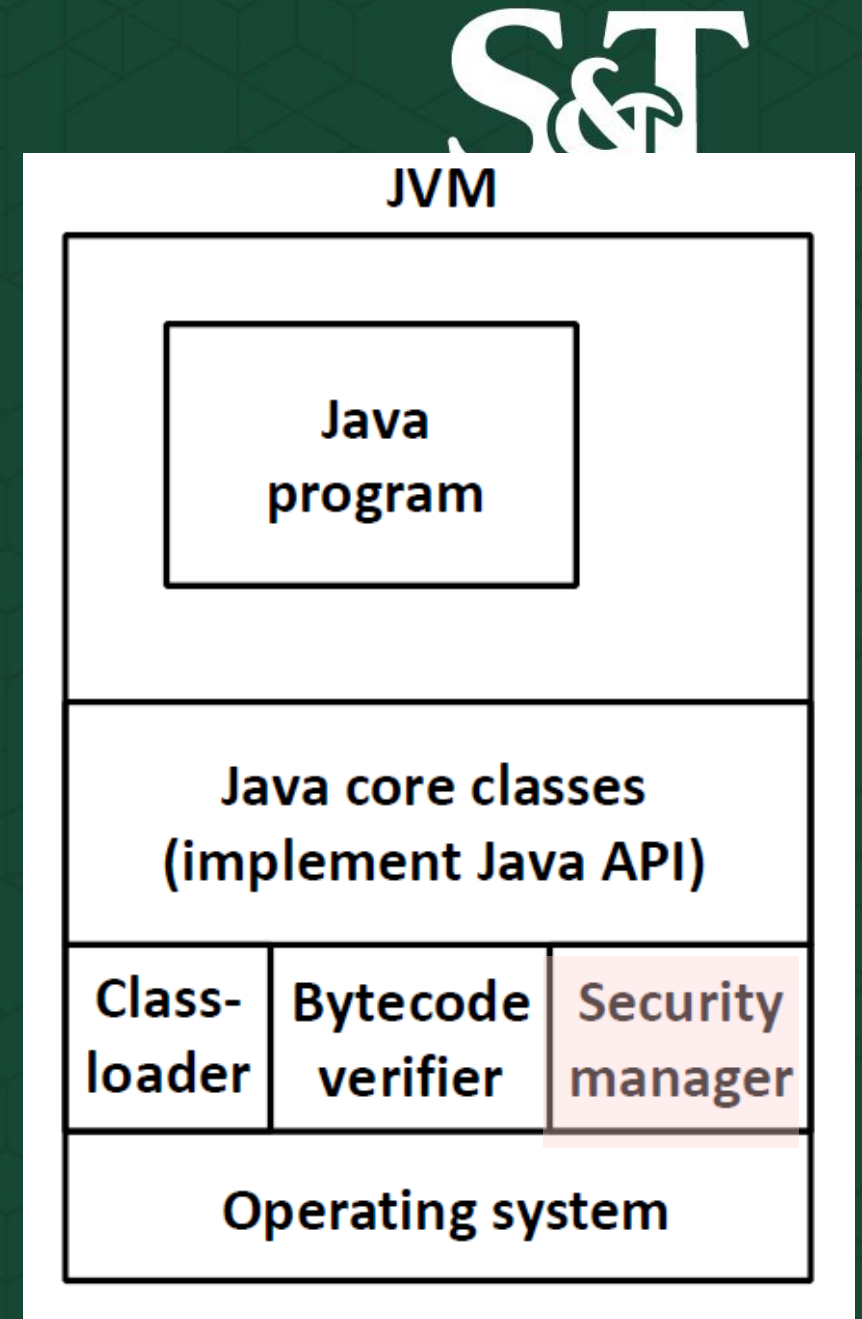


MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- What untrusted code can do
 - Access CPU and memory to build its objects and execute
 - Connect to the web server from which the applet was downloaded
- What untrusted code can't do
 - Operations on client's file system (read, write, delete, ...)
 - Connect to destinations other than its origin
 - Make critical system calls, such as `System.exit()`
 - Create a classloader or security manager
 - Other dangerous actions, e.g., manipulate other threads

Stack Inspection in Java

- How does security manager evaluate a request?
- Example:
 - The code requests to access the file system.
 - Stack inspection.



Stack Inspection Operations



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
—— ROLLA, MISSOURI ——

- **enablePrivilege(*P*)**
 - When code wants to use a permission *P*, it must first call **enablePrivilege(*P*)**
 - If permitted, the current stack frame is annotated with an enabled privilege mark
- **revertPrivilege(*P*)**
 - Removes annotation from stack frame
- **disablePrivilege(*P*)**
 - Creates a stack annotation to hide the earlier enabled privilege
- **checkPrivilege(*P*)**
 - Search stack from the newest to the oldest to check if a frame has proper privilege to make a specific system call

Simplified Stack Inspection



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Assume two principals (“system” & “untrusted”) and one privilege**
- **Each Java thread has a run-time stack that tracks method calls**
- **Every stack frame is labeled with principal and a privilege tag**
 - A system class can set the tag while untrusted code cannot
 - Whenever a frame completes its work, the tag disappears
- **Per access request**
 - Algorithm searches frames from newest to oldest
 - If a frame with “full” tag is first found, then access is permitted
 - If an untrusted frame is encountered, access is denied

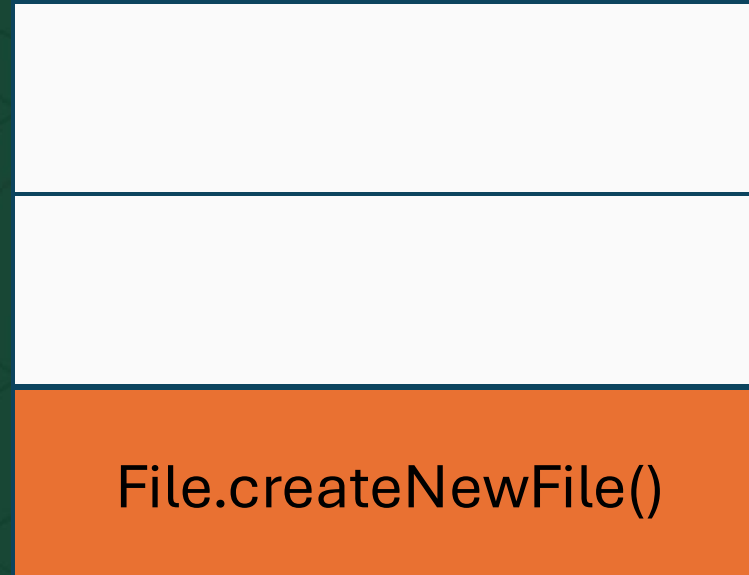
Simplified Stack Inspection



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

...

Stack grows



checkPrivilege()

Case I



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

`enablePrivilege()`

trusted code

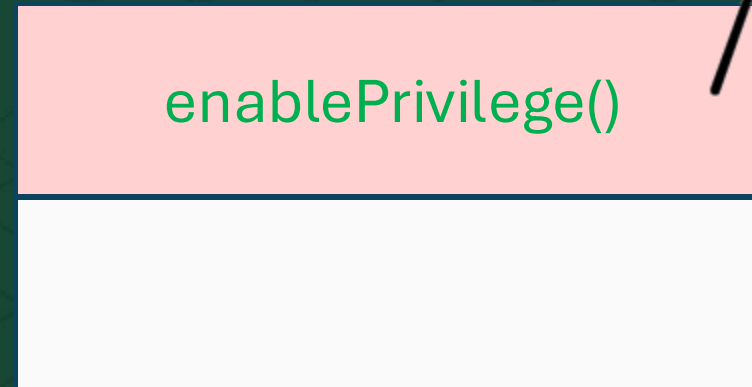
Stack grows



Case I



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI



trusted code

Stack grows



Case I



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



enablePrivilege()

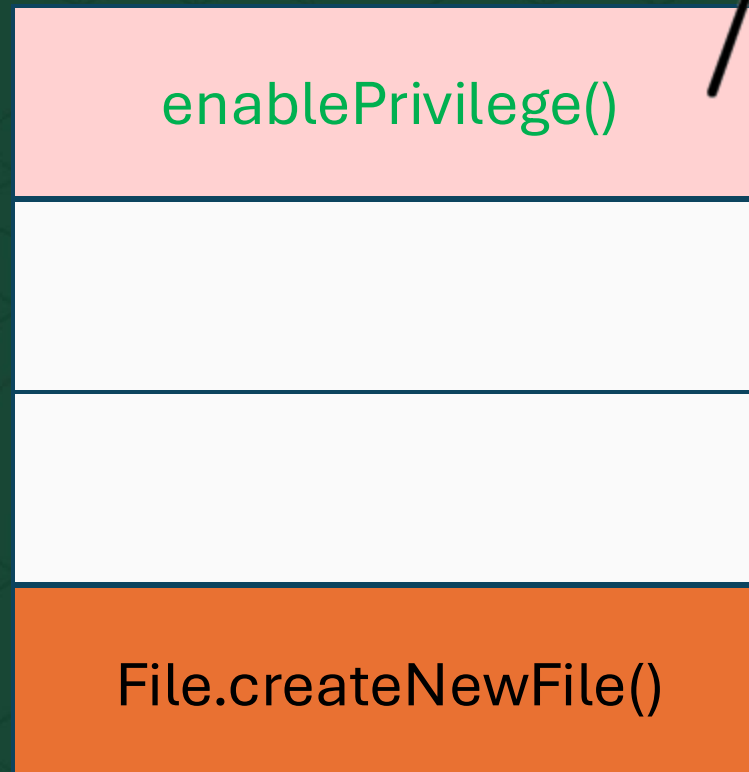
trusted code

Case I



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



checkPrivilege()

trusted code

Look up!

First flag encountered is “enable” → access is allowed!



Case II



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

`enablePrivilege()`

trusted code

Stack grows

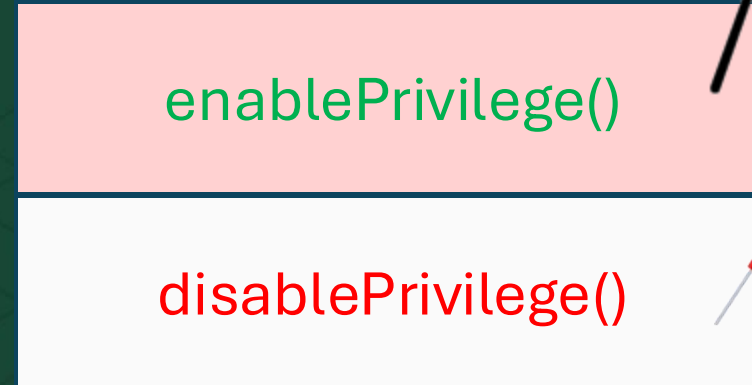


Case II



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



trusted code

Case II



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



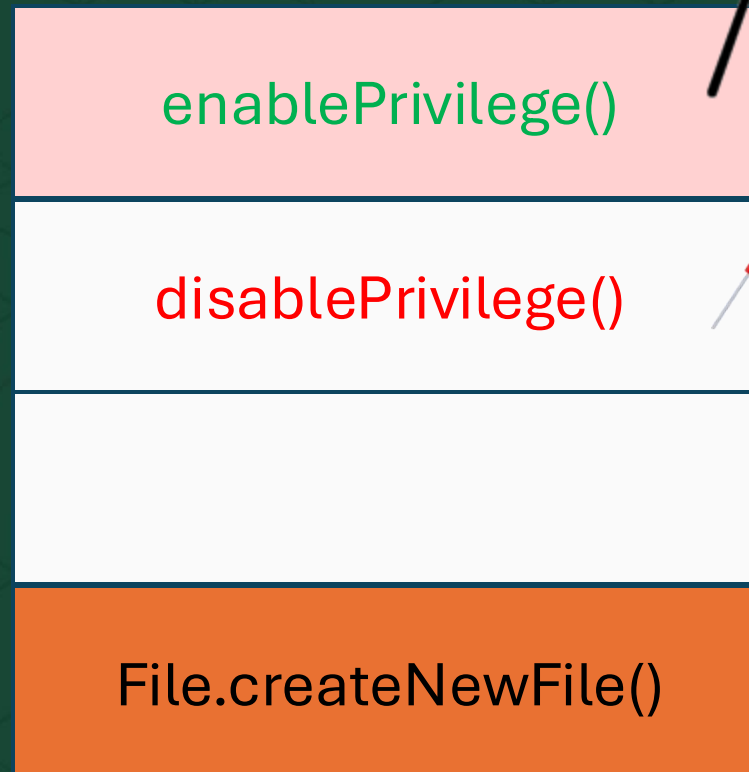
trusted code

Case II



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



checkPrivilege()



trusted code



Look up!

First flag encountered is “disable” → access is denied!

Case III



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

enablePrivilege()



trusted code

Stack grows

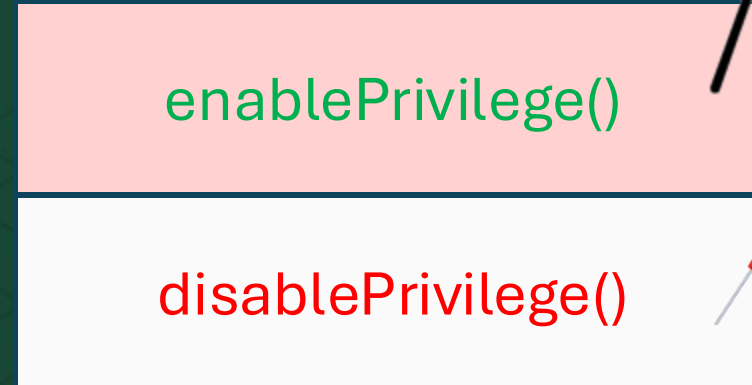


Case III



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



trusted code

Case III



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



trusted code

Case III



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



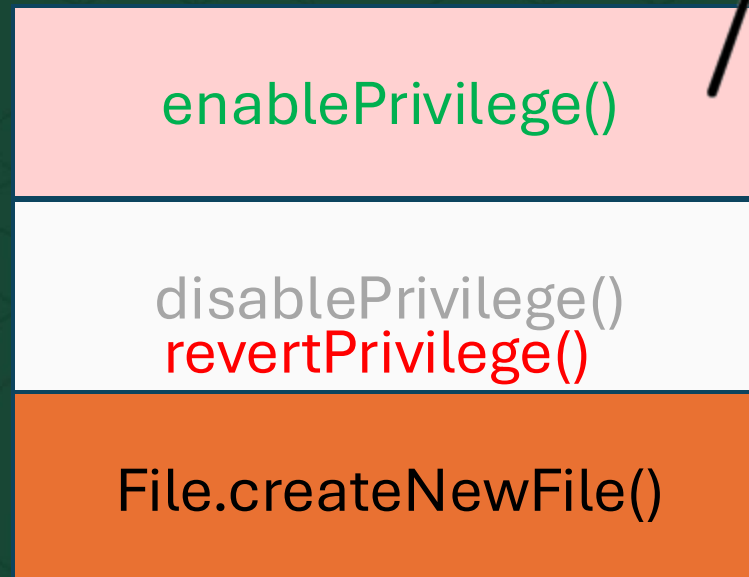
trusted code

Case III



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI

Stack grows



checkPrivilege()

trusted code



First flag encountered is “enable” → access is granted!

Stack Inspection Algorithm

```
checkPrivilege (target) {  
    // loop, newest to oldest stack frame  
    foreach stackFrame {  
        if (local policy forbids access to target by class  
            executing in stackFrame)  
            throw ForbiddenException;  
        if (stackFrame has enabled privilege for target)  
            return; // allow access  
        if (stackFrame has disabled privilege for target)  
            throw ForbiddenException;  
    }  
    // if we reached here, we fell off the end of the stack  
    if (Netscape 4.0)  
        throw ForbiddenException;  
    if (Microsoft IE 4.0 || Sun JDK 1.2)  
        return; // allow access  
}
```



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

Simplified Stack Inspection



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Assume two principals (“system” & “untrusted”) and one privilege**
- **Each Java thread has a run-time stack that tracks method calls**
- **Every stack frame is labeled with principal and a privilege tag**
 - A system class can set the tag while untrusted code cannot
 - Whenever a frame completes its work, the tag disappears
- **Per access request**
 - Algorithm searches frames from newest to oldest
 - If a frame with “full” tag is first found, then access is permitted
 - If an untrusted frame is encountered, access is denied

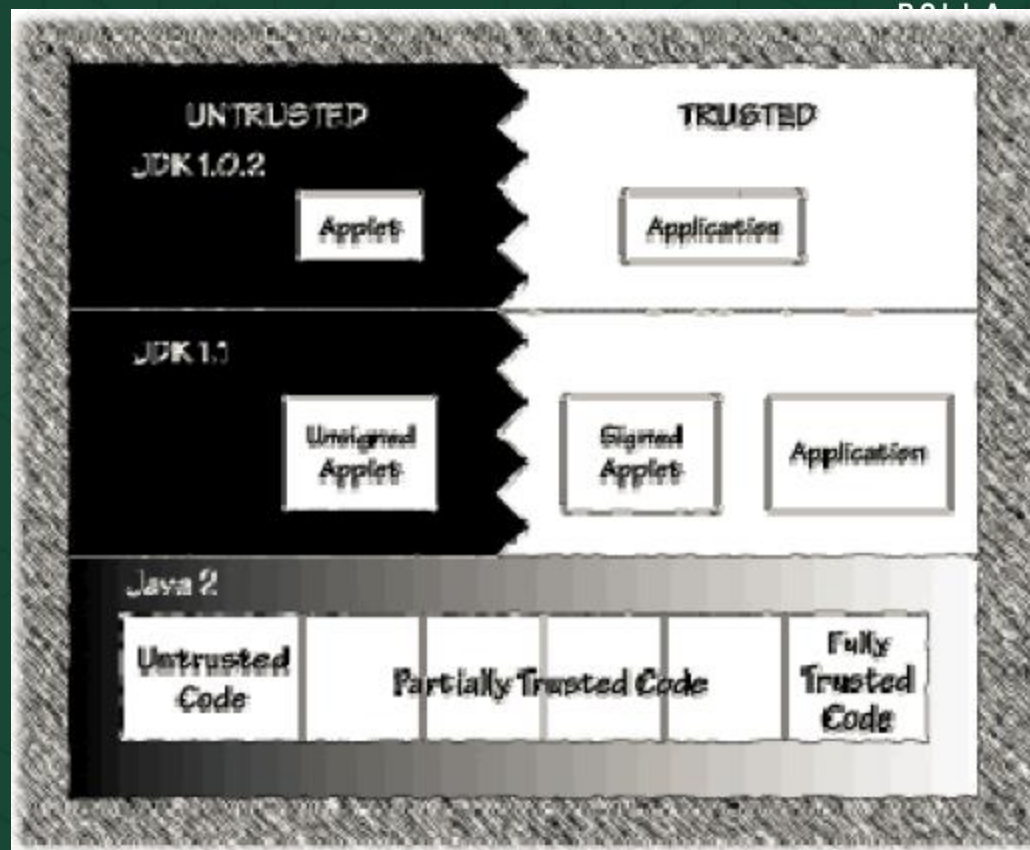
Security Manager Trust Model



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY

ROLLA, MISSOURI

- Trust model has become increasingly refined
- **JDK 1.0.2**
 - Untrusted = any applet
 - Trusted = application
- **JDK 1.1**
 - Untrusted = unsigned applet
 - Trusted = signed applet or application
- **JDK 1.2 and beyond**
 - Many shades of gray



Java 2 Security Policy



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Identity**
 - In Java 2, there are two identity-defining characteristics:
 - **Origin** (where the code comes from) and
 - **Signature** (who vouches for it)
 - Origin and Signature are represented with
 - **java.security.CodeSource**
- **Permissions (java.security.Permission)**
 - Permissions include:
 - **java.io.FilePermission** for file system access
 - **java.net.SocketPermission** for network access
 - **java.lang.PropertyPermission** for Java properties
 - **java.lang.RuntimePermission** for access to runtime system
 - **java.security.NetPermission** for authentication
 - **java.awt.AWTPermission** for access to graphical resources
 - An example of a file permission:
 - **p = new FilePermission("/applets/tmp/scratch","read");**

Java 2 Security Policy



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

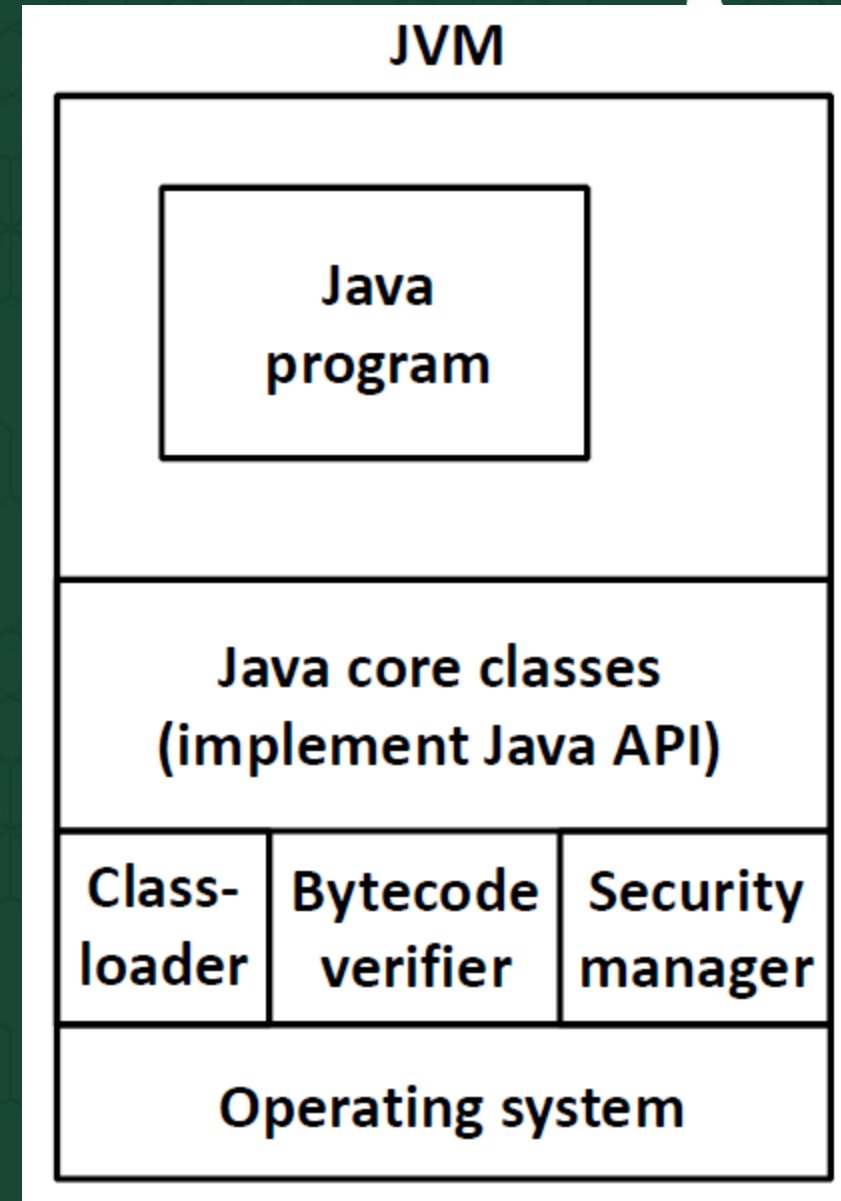
- Policy: A mapping from identity to permissions
 - The policy object is a runtime representation of policy usually set up by the VM at startup time (much like the Security Manager). E.g.,

```
grant CodeBase "https://www.rstcorp.com/users/gem",  
    SignedBy "*" {  
    permission java.io.FilePermission "read,write",  
        "/applets/tmp/*";  
    permission java.net.SocketPermission "connect",  
        "*.rstcorp.com";  
};
```

- Sun Java 2 policy can be set by users (bad idea) or system administrators
- Permissions are additive

JVM Wrap Up

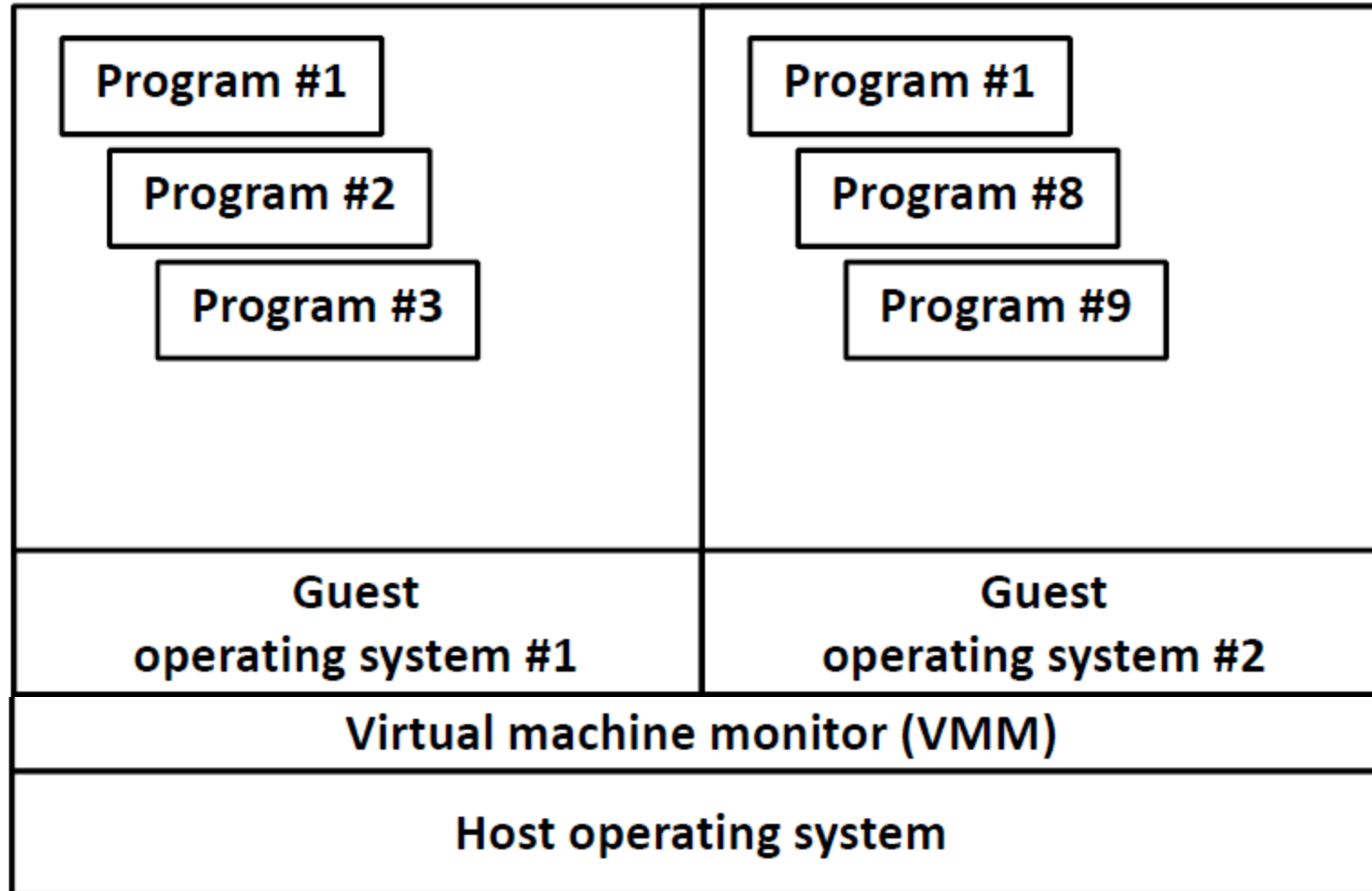
- **Type safety provides memory protection**
 - Between Java programs in a VM
 - Between Java programs and VM itself
- **Security manager controls how Java programs interact with resources outside VM**
 - Different programs need different levels of privilege
 - Non-trivial to decide which accesses to protected resources should be allowed
 - Hard to tell who is asking and why



Sandbox #2: VM Monitors



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
ROLLA, MISSOURI



Typical goal: Complete isolation between guest OSes

VMM Security Assumptions



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **VMM assumptions**
 - Malware can infect guest OS and guest apps
 - Malware cannot escape from the infected VM
 - Cannot infect Host OS
 - Cannot infect other VMs on the same hardware
- **Requires that VMM protect itself and is not buggy**
 - VMM is much simpler than full OS
 - VMM is able to inspect guest OS without its consent or knowledge.
- **Host OS shouldn't run applications directly!**

Example Use #1

Intrusion Detection / Anti-virus



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Runs as part of OS kernel and user space process**
 - Kernel root kit can shutdown protection system
 - Common practice for modern malware
- **Standard solution: run IDS system in the network**
 - Problem: insufficient visibility into user's machine
- **Better: run IDS as part of VMM (protected from malware)**
 - VMM can monitor virtual hardware for anomalies
 - VMI: Virtual Machine Introspection
 - Allows VMM to check Guest OS internals

Example Use #1

Sample Checks



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Stealth malware**

- Creates processes that are invisible to “ps”
- Opens sockets that are invisible to “netstat”

1. Lie-detector check

- Goal: detect stealth malware that hides processes and network activity
- Method:
 - VMM lists processes running in guest OS
 - VMM requests guest OS to list processes (e.g., ps)
 - If mismatch, kill VM

Example Use #1

Sample Checks



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

2. Application code integrity detector

- VMM computes hash of user app-code running in VM
- Compare to whitelist of hashes
 - Kills VM if unknown program appears

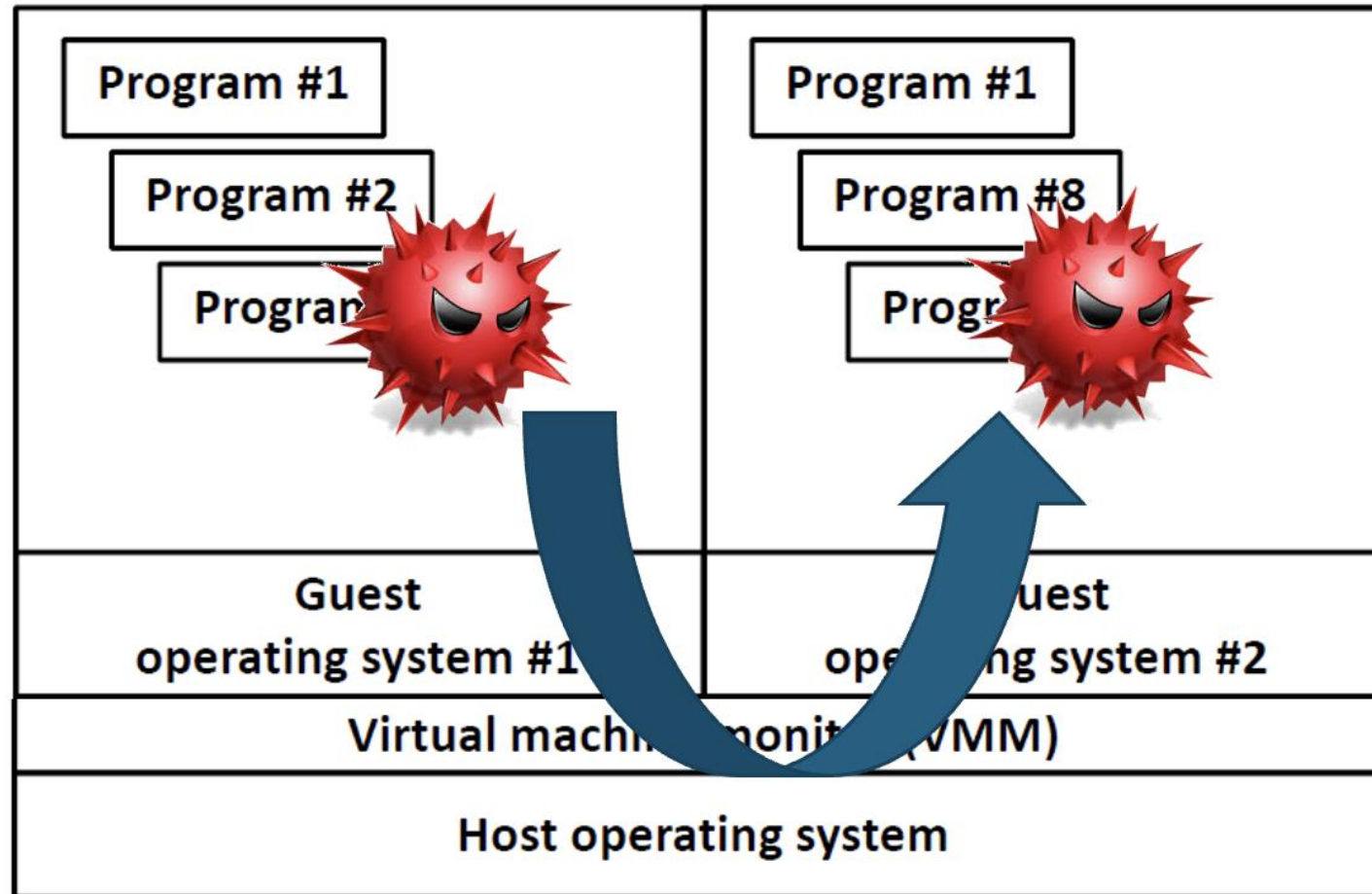
3. Ensure guest OS kernel integrity

- Example: detect changes to `sys_call_table`

4. Virus signature detector

- Run virus signature detector on guest OS kernel memory

Can two completely isolated malwares communicate under VMM?



Example Problem #1

An Example Covert Channel



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Both VMs use the same underlying hardware**
- **To send a bit b in $\{0,1\}$ malware does:**
 - $b = 1$: at 1:30am do CPU intensive calculation
 - $b = 0$: at 1:30am do nothing
- **At 1:30.00am listener does a CPU intensive calculation and measures completion time**
 - Now $b = 1$ if completion-time $>$ threshold
- **Many covert channels exist in running system**
 - File-lock status, cache contents, interrupts, ...
 - Very difficult to eliminate

Example Problem #2

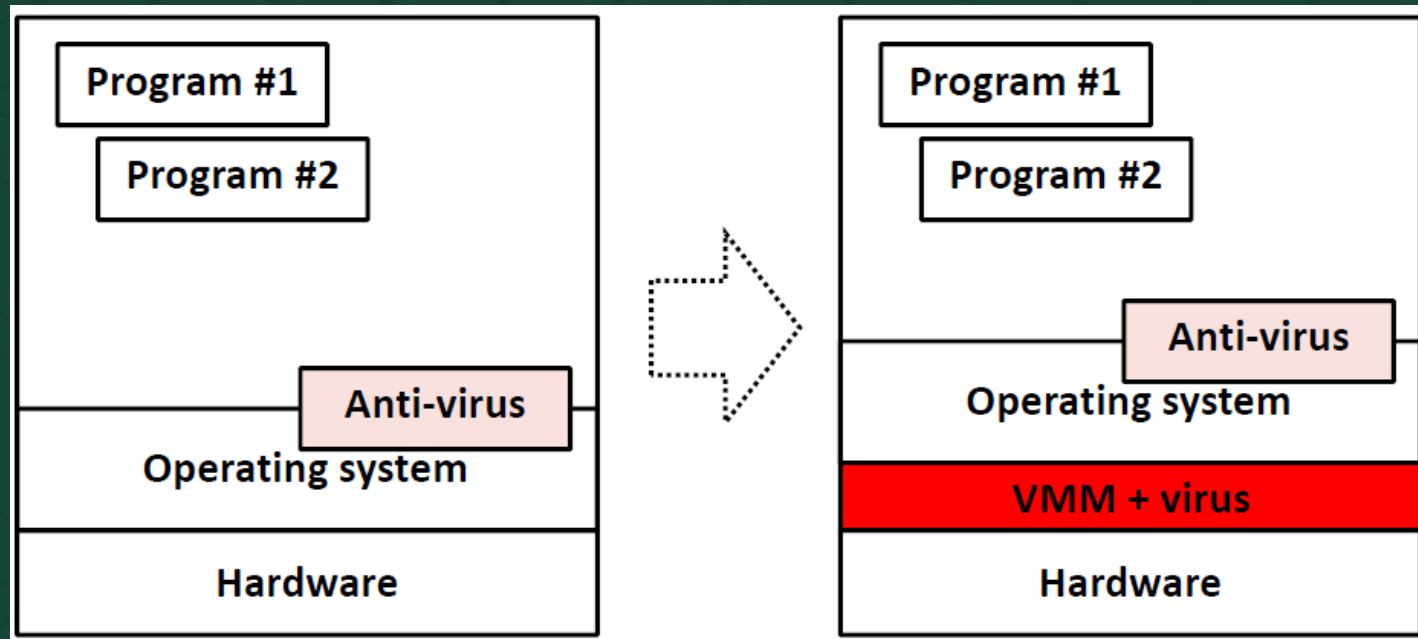
VM Based Malware



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Virus idea:**

- Once on the victim machine, install a malicious VMM
- Virus hides in VMM
- Invisible to virus detector running inside VM



Example Problem #2

VM Based Malware



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- VM-based rootkit (VMBR): a virus that installs a malicious VMM.
- **Microsoft Security Bulletin, Oct 2006:**
 - <http://www.microsoft.com/whdc/system/platform/virtual/CPUVirtExt.mspx>
 - Suggests disabling hardware virtualization features by default for client-side systems

VMM Detection



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **Can an OS/application detect it is running on top of a VMM?**
- **Applications (a.k.a. why important?):**
 - Virus detector can detect VMBR
 - Normal virus (non-VMBR) can detect VMM
 - refuse to run to avoid reverse engineering
 - Software that binds to hardware (e.g., MS Windows) can refuse to run on top of VMM
 - DRM systems may refuse to run on top of VMM

VMM Detection



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **VM platforms often emulate simple hardware**
 - VMWare emulates an ancient i440bx chipset
 - .. but report 8GB RAM, i7 CPUs, etc.
 - **VMM introduces time latency variances**
 - Memory cache behavior differs in presence of VMM
 - Results in relative latency in time variations for any two operations
 - **VMM shares the translation lookaside buffer (TLB) with GuestOS**
 - GuestOS can detect reduced TLB size
- ... and many more methods**

VMM Wrap Up



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- **VMs can help provide confinement/isolation**
- **But...**
- **... the perfect VMM does not exist**
- **VMMs today (e.g., VMWare) focus on:**
 - Compatibility: ensure off the shelf software works
 - Performance: minimize virtualization overhead
- **VMMs do not provide transparency**
 - Anomalies reveal existence of VMM

Summary



MISSOURI UNIVERSITY OF
SCIENCE AND TECHNOLOGY
— ROLLA, MISSOURI —

- Two new approaches to provide isolation
 - Java sandboxing, type safety, security policy and inspection, ...
 - VMMs, issues, covert channels.